JavaScript

- It is used to add interactivity inside html web pages to perform client side validations , ajax call to refresh page data
- Need to run page in web browser only as web browser interpreted JS code and then eveluate and run with help of JavaScript Engine
- JavaScript Engine read js code and convert inside browser code so browser can understand and eveluate

- Every Browser has different JS Engine

  Chrome / Edge – V8

  Firefox - Spider Monkey

  Safari – Apple

  So when we are writing JS Code it is not execute same on every browser as JS engine is different

  Then Developer's need to take care to write extra code for Browser Compability

Features of JavaScript

- Object Oriented
- Platform Independent
- Update HTML using DOM
- Capable of Exception Handling
- Allows Asynchronous Functions

- Single Threaded - Executing one task at a time

Requirement

- As we are in Internet World we required application need to execute fast as well as multi - tasking

- Application should be scalable in terms of number of users

- To Fulfill that Requirement we required something new nothing but Node JS

- V8 JavaScript Engine is opensource so Scientiest **Ryan** took that Engine Code add embedded C++ Language with that and comes with new Concept nothing but **Node JS**

- V8 – JS Engine --- using internally to Browser
- C++ having machine functionality to deal with file handling ,

- NODE JS is runtime Env for JS , So we can run outside browser

- It is a non-blocking, event-driven architecture that allows businesses to handle high volumes of concurrent requests effectively, which is why companies like Netflix have reduced startup time by 70% after adopting Node.js.

Why Node JS

- It Enables the use of JavaScript for both frontend and backend development.
- It Supports building real-time applications like chat apps and gaming servers.
- It Provides high scalability for I/O-heavy applications.
- Non-blocking, event-driven architecture for high performance.
- Supports the creation of REST APIs, real-time applications, and microservices
- It Backed by a vibrant community and extensive library support

- **High Performance: Built** on Chrome's V8 engine, NodeJS compiles JavaScript to machine code, ensuring fast execution, especially for I/O-bound tasks.
- **Single-Threaded Event Loop:** Uses a non-blocking, event-driven model to handle many concurrent requests efficiently with minimal memory usage.
- **Scalability:** Ideal for scalable applications, NodeJS can handle high traffic by scaling horizontally and vertically.
- **Unified Language:** JavaScript is used both on the client-side and server-side, streamlining development.
- **Rich Ecosystem (npm):** Access to a vast collection of libraries and tools via npm, simplifying development.
- **Real-Time Capabilities:** Perfect for real-time applications like chat and live updates.
- **Cross-Platform:** Works across different operating systems (Windows, macOS, and Linux).

Browser JS	Node JS
Runs within Browser and provide DOM Object to manipulate UI	It Provide Env to run JS Outside Browser , no support for DOM
Support Manipulation of GUI with help of DOM	No GUI Support as it is run on server Side no direct connection with user , we can use CLI to run node js Code

Feature	Node.js	Browser
File System Access	Yes	No
Networking (TCP/UDP)	Yes	No
DOM Access	No	Yes
Global Object	global	window/self
Modules	CommonJS/ESM	ESM/Scripts
Environment Variables	Yes (process.env)	No
Security	Full OS access	Sandboxed
Package Management	npm/yarn	CDN/Bundle

Node JS Enviornment

Download and Install Node.js

- Go to https://nodejs.org
- Download the **LTS (Long Term Support)** version
- Run the installer and follow the instructions

- After Installation open cmd and type node –v

Will get version of Node JS

V8 Engine

Node JS always use V8 Engine from Browser

- The V8 engine is Google's open-source JavaScript engine, used by Chrome and Node.js.
- It compiles JavaScript to native machine code for fast execution.

1. **Origin:** Developed by Google for Chrome in 2008
2. **Integration:** Node.js uses V8 to provide JavaScript runtime on the server

Features: ES6+ support

- **Just-In-Time (JIT) Compilation:** Converts JavaScript into optimized machine code instead of interpreting it
- **Hidden Classes:** Optimizes property access on JavaScript objects
- **Efficient Garbage Collection:** Manages memory to prevent leaks and optimize performance
- **Inline Caching:** Speeds up property access by remembering where to find object properties

ECMAScript

- **JavaScript**, invented by Brendan Eich in 1995, became an **ECMA** standard in 1997.
- **ECMAScript** is the official name of the **JavaScript standard**.
- **From 1997** ECMA versions was abbreviated by numbers. (ES1, ES2, ES3, ES5, ES6).
- All latest features are in ES6

  arrow function , … operator etc

- Ecma International (formally European Computer Manufacturers Association)

- It is a non-profit organization that develops standards in computer hardware, communications, and programming languages.11

- Client always making Request to Server

Client -------Request ---------→Server

Request can be Synchronous or Asynchronous (Blocking / Non Blocking)

All Request need to be add inside Event Queue

- Queue - First In First Out

- If the request is **non-blocking**, it is processed immediately, and the response is sent back to the client. If the request is a **blocking operation**, it is passed to the **Thread Pool**.
- The Thread Pool has a **limited number of threads.** If a thread is free, the blocking task is assigned to it.

- Once the task is completed, the thread returns the result back to the Event Loop, which then sends the response to the client.
- If all threads are busy, any new blocking requests must **wait** in a queue until a thread becomes free.
- This waiting increases the **response time** for the client. Therefore, it is always better to **avoid blocking operations** and use **non-blocking alternatives** whenever possible to maintain performance and scalability.

Demo of Node JS

- Make sure Node js installed on System

node –v /// will print node js version

1)Create Folder where you want to save js files

2) Open command Prompt with same path where js files are kept

3) Node <file name>.js

Check syntax without executing
node --check app.js

- If we not having .js file then directly we can execute code on command Prompt (REPL)

1) Type node and press enter

NodeJS REPL (Read-Eval-Print Loop)

- It is an interactive shell that allows you to execute JavaScript code line-by-line and see immediate results.
- This tool is extremely useful for quick testing, debugging, and learning, providing a sandbox where you can experiment with JavaScript code in a NodeJS environment.
- **READ:** You type some JavaScript code into the terminal, and REPL reads what you typed.
- **EVAL:** REPL runs (evaluates) your code.
- **PRINT**: REPL shows you the result of your code.
- **LOOP**: REPL goes back to step 1, waiting for you to type more code.This loop continues until you quit REPL.

- We can write and eveluate js code using REPL

- UNDERSCORE (_) is used to store output of 1st expression and use it in another
- 12*24
- _ / 12

- To store all commands in js file we can use
- .save <filename>.js

- **Built-in REPL Commands**
- NodeJS REPL provides several built-in commands (REPL commands always start with a dot .).
- **.help:** Displays a list of all available REPL commands.
- **.break:** Breaks out of multi-line input or clears the current input.
- **.clear:** Resets the REPL context by clearing all declared variables.
- **.exit:** Exits the REPL session.

Non blocking feature

- Non-blocking approach enables Node.js to handle thousands of concurrent connections efficiently.

- It is possible due to **Asynchronous**

Programming

Call back

Promise

Async/Await

Call Back

- A callback function is a function that is passed as an argument to another function and executed later.
- A function can accept another function as a parameter.
- Callbacks allow one function to call another at a later time.
- A callback function can execute after another function has finished.

Callback Function

- function <function Name>

```
                    (param1, param2, callback) {
                              // Do some work...
                              // Call the callback function
                        callback(result);
}
```

Handling Asynchronous Operations

Callbacks are widely used in

- API requests (fetching data)
- Reading files (Node.js file system)
- Event listeners (clicks, keyboard inputs)
- Database queries (retrieving data)

Promises

- JavaScript Promises make handling asynchronous operations like API calls, file loading, or time delays easier. Think of a Promise as a placeholder for a value that will be available in the future. It can be in one of three states
- **Pending:** The task is in the initial state.
- **Fulfilled:** The task was completed successfully, and the result is available.
- **Rejected:** The task failed, and an error is provided.

- *let promise = new Promise((resolve, reject) =>*
  {
  // Perform async operation
  if (operationSuccessful) {
  resolve("Task successful");
  } else {
  reject("Task failed");
  }
  });

What is the Event Loop?

- The **event loop** is what makes Node.js non-blocking and efficient.
- It handles asynchronous operations by delegating tasks to the system and processing their results through callbacks, allowing Node.js to manage thousands of concurrent connections with a single thread.
-

Node.js follows these steps to handle operations:

- Execute the main script (synchronous code)
- Process any microtasks (Promises, process.nextTick)
- Execute timers (setTimeout, setInterval)
- Run I/O callbacks (file system, network operations)
- Process setImmediate callbacks
- Handle close events (like socket.on('close'))

- Event Loop Phases
- The event loop processes different types of callbacks in this order:
- **Timers**: setTimeout, setInterval
- **I/O Callbacks**: Completed I/O operations
- **Poll**: Retrieve new I/O events
- **Check**: setImmediate callbacks
- **Close**: Cleanup callbacks (like socket.on('close'))

Why is the Event Loop Important?

- The event loop enables Node.js to handle thousands of concurrent connections with a single thread, making it perfect for:
- Real-time applications
- APIs and microservices
- Data streaming
- Chat applications